

Conclusion

COGANT frames codebase analysis as a pipeline from ingestion through a program graph IR to **Generalized Notation Notation (GNN)** exports, with explicit confidence and provenance so that learning systems can treat analysis noise as data rather than as hidden failure modes. The Python layer provides session and pipeline APIs, a bundle abstraction, CLI, review tooling, and HTML reporting; the Rust layer concentrates graph mechanics and export formatting under crate boundaries described in [../cogant/docs/architecture/README.md](https://cogant/docs/architecture/README.md).

Shipped Capabilities

Several capabilities ship in the current v0.5.0 release and together define the behaviour that downstream users can rely on:

1. **Fixpoint translation engine with 19 real rules.** The shipped `TranslationEngine` re-applies every registered rule on each pass, terminates as soon as a pass produces zero new mappings, and bounds pathological rule sets with a configurable iteration cap. Each iteration boundary is recorded in the internal match log, so convergence can be